

ECOM EXPERTS

Ecom errores

Manual de usuario, técnico y de desarrollo

ecom_errores 1.1.0 · 07/09/2026

Contenido

Qué hace

Instalación

Configuración

Captura TOTAL (antes de que arranque nada) — IMPORTANTE

Cómo diagnosticar tu error de carritos

Ver y limpiar los logs

Preguntas frecuentes

Objetivo

Mecanismo de captura

Registro ultra-temprano (dos fases)

Fase prepend (captura TOTAL, antes del arranque)

Fase runtime (refuerzo dentro de PrestaShop)

Filtrado de ruido

Detección de módulo

Formato del log

Rotación y seguridad

Claves de Configuration

Compatibilidad

Estructura

Flujo de arranque

Puntos de extensión

Uso manual desde otro código

Decisiones de diseño

Pruebas recomendadas

Estándares aplicados

Qué hace

Vigila tu tienda y, cuando ocurre un **error fatal** o una excepción que rompe la página, lo apunta en un fichero de texto dentro de `modules/ecom_errores/logs/`. De cada fallo guarda: la fecha, el **fichero y la línea**, **qué módulo** lo provocó, la URL, y el **estado del carrito y cliente** en ese momento. No hace caso a los avisos de caché, plantillas (`.tpl`), `var` ni compilados.

Instalación

1. Sube el módulo `ecom_errores` (ZIP completo).
2. En **Módulos**, instálalo. Se activa la captura automáticamente.
3. Entra en **Configurar**.

Configuración

- **Activar captura:** enciende/apaga el registro.
- **Nivel de captura:**
 - *Solo fatales:* los más graves.
 - *Fatales + excepciones* (recomendado): lo que normalmente causa la pantalla en blanco / 500.
 - *Todo:* incluye warnings y notices. Muy ruidoso, úsalo solo un rato para depurar.
- **Guardar contexto de carrito:** añade `id_cart`, nº de productos, `id_customer` e `id_guest`. **Déjalo activado:** es la clave para tu fallo que solo aparece cuando hay carritos.
- **Patrones a ignorar:** rutas que se descartan (por defecto `var`, `cache`, `smarty`, `.tpl`, `compile`).
- **Tamaño máximo por log:** cuando el fichero crece demasiado, se rota automáticamente.

Captura TOTAL (antes de que arranque nada) — IMPORTANTE

Si tu error fatal aparece **antes de que se ejecute nada** (pantalla en blanco / 500 nada más entrar), los hooks normales no llegan a tiempo. En la configuración, en el panel "**Monitorización TOTAL**", pulsa **Activar monitorización**. Esto hace que un vigilante se ejecute en cada petición desde el primer instante. El panel te dice si tu servidor usa el método `.user.ini` (botón verde) o si necesitas pegar una línea en el `.htaccess` (te la muestra). Puede tardar hasta 5 minutos en aplicar. Si algo fuera mal, pulsa **Desactivar** (o borra esa línea) y la web vuelve a la normalidad; el vigilante está blindado y nunca rompe la tienda.

Cómo diagnosticar tu error de carritos

1. Deja el nivel en **Fatales + excepciones** y el contexto de carrito **activado**.
2. Reproduce el fallo (añade productos al carrito y haz la acción que revienta).
3. Vuelve a la configuración del módulo: abajo verás las **últimas líneas del log**.

4. Busca el bloque más reciente. Te dirá el **Módulo**, el **Fichero**, la **Línea** y el **Backtrace**. El apartado *Contexto carrito/cliente* confirma con qué carrito ocurrió.
5. Envía ese bloque a tu desarrollador (o pégalo aquí) para el arreglo.

Ver y limpiar los logs

En la configuración tienes: - **Vaciar logs**: borra todos los ficheros de error. - **Escribir prueba**: comprueba que el módulo puede escribir en `/logs`. - Una tabla con los ficheros y su tamaño, y un cuadro con las últimas líneas.

Los ficheros están en `modules/ecom_errores/logs/error-AAAA-MM-DD.log` (uno por día) y **no son accesibles desde el navegador** por seguridad; se leen por FTP o desde el panel del módulo.

Preguntas frecuentes

¿Ralentiza la tienda? No: solo actúa cuando ocurre un error real. **¿Puede romper algo?** No: si el propio registro fallara, se ignora en silencio y la tienda sigue funcionando como siempre. **No veo nada en el log.** Asegúrate de que "Activar captura" está en Sí, reproduce el error, y recuerda que los fallos de caché/tpl se ignoran a propósito.

Ecom Errores — Documento Técnico

Objetivo

Capturar los errores fatales y excepciones no controladas de PrestaShop 8/9 **antes de que la página muera**, guardando en `/logs` el fichero, la línea, el módulo culpable y el contexto de carrito/cliente. Ignora el ruido de `var`, `cache`, `smarty`, `.tpl` y `compile`.

Mecanismo de captura

Se registran tres manejadores complementarios (en `EcomErroresLogger::init()`):

1. `set_exception_handler` → captura `Throwable` no atrapados (`TypeError`, `Error`, `Exception`). En PS 8/9 la mayoría de "errores fatales" son en realidad excepciones no capturadas, así que aquí obtenemos **backtrace completo** y se puede identificar el módulo con precisión. Se **encadena** con el handler previo de PrestaShop (`self::$prevException`) para no romper la página de error nativa.
2. `register_shutdown_function` → captura fatales reales (`E_ERROR`, `E_PARSE`, `E_COMPILE_ERROR`, función inexistente, memoria agotada) leyendo `error_get_last()`. Es aditivo, no reemplaza nada.
3. `set_error_handler` → errores de usuario / recuperables y, con nivel `all`, warnings y notices. También se encadena con el handler previo.

Registro ultra-temprano (dos fases)

Para capturar un fatal que ocurre **antes de ejecutarse nada** (core de PrestaShop u otro módulo que carga antes), los hooks del módulo llegan tarde. Por eso hay dos fases:

Fase `prepend` (captura TOTAL, antes del arranque)

El fichero raíz `bootstrap.php` se ejecuta vía `auto_prepend_file` (PHP lo carga antes que cualquier código de PrestaShop). Llama a `EcomErroresCore::init(null, 'prepend')`, que lee la config del fichero plano `config.json` (no hay DB todavía) y registra shutdown + exception + error handlers desde el byte 0. El log marca `Fase PS: NO cargado` cuando el fatal es de esta ventana. `bootstrap.php` y `classes/ErrorCode.php` **no** llevan el guard `_PS_VERSION_` (a propósito: deben correr sin PrestaShop; su acceso web está bloqueado por `index.php` / `.htaccess`).

Se activa desde el back-office (botón "Activar monitorización") escribiendo un bloque marcado en el `.user.ini` de la raíz, o manualmente vía `.htaccess` (`mod_php`) / `php.ini`.

Fase `runtime` (refuerzo dentro de PrestaShop)

En el constructor y en los hooks `actionDispatcherBefore`, `actionFrontControllerSetMedia` y `displayBackOfficeHeader`, se llama a `init($configDB, 'runtime')`, que **reinstala** nuestro handler por encima del de PrestaShop y **encadena** con el suyo (captura fatales tardíos sin romper la página 500). Sirve también como fallback si no se activó el `auto_prepend`.

`init()` usa guards separados (`$shutdownDone` , `$earlyDone` , `$lateDone`): el shutdown es único y aditivo; las fases `prepend` y `runtime` registran una vez cada una.

Filtrado de ruido

`shouldIgnore($file)` normaliza la ruta (barras + minúsculas) y descarta si contiene alguno de los patrones configurables (`ECOM_ERRORES_IGNORE` , uno por línea). Por defecto: `/var/` , `/cache/` , `smarty` , `.tpl` , `compile` .

Para excepciones con traza, `pickOrigin()` busca el **primer frame real** (no ignorado); si toda la pila es ruido, el error se descarta. Así un fallo cuyo frame superior está en `var/cache` pero que se origina en un módulo se atribuye correctamente al módulo.

Detección de módulo

`detectModule()` escanea la ruta de origen, el fichero del error y todos los frames del backtrace con la regex `#/modules/([^\s]+)/#` , ignorando `ecom_errores` . Si no encuentra ninguno devuelve (`nucleo/tema, no es un modulo`) .

Formato del log

Ficheros diarios `logs/error-YYYY-MM-DD.log` . Cada registro incluye: fecha/hora, tipo, mensaje, fichero, línea, módulo, URL, método HTTP, IP, pico de memoria, contexto de carrito y backtrace.

Contexto de carrito (`ECOM_ERRORES_CART_CONTEXT`): `id_cart` , nº productos, `id_customer` , `id_guest` — leído de forma defensiva vía `Context::getContext()` , envuelto en `try/catch` para no provocar más errores.

Rotación y seguridad

- Rotación por tamaño (`ECOM_ERRORES_MAXSIZE` KB, 5 MB por defecto) → renombra a `.His.old` .
- `logs/.htaccess` bloquea descarga web de `.log` / `.old` .
- `index.php` en cada directorio con `header('Location: ../'); exit;` .
- Todo el logger está blindado con `try/catch` (`Exception` + `Throwable`): **nunca** rompe el request por sí mismo.

Claves de Configuration

Clave	Valor por defecto	Descripción
<code>ECOM_ERRORES_ENABLED</code>	1	Activa/desactiva la captura
<code>ECOM_ERRORES_LEVEL</code>	<code>fatal_exceptions</code>	<code>fatal</code> / <code>fatal_exceptions</code> / <code>all</code>
<code>ECOM_ERRORES_IGNORE</code>	var, cache, smarty, .tpl, compile	Patrones a ignorar (1/línea)
<code>ECOM_ERRORES_CART_CONTEXT</code>	1	Adjunta contexto de carrito
<code>ECOM_ERRORES_MAXSIZE</code>	5120	Tamaño máx. KB antes de rotar

Compatibilidad

PHP 7.4 y 8.x. Sin Composer, sin namespaces en el fichero principal, `require_once` manual, rutas relativas a `_PS_MODULE_DIR_`, `\Context::getContext()`, sin `::class`.

Ecom Errores — Documento de Desarrollo

Estructura

```
ecomErrores/  
├─ ecomErrores.php      Clase principal EcomErrores (hooks, install, config)  
├─ classes/  
│   ├─ ErrorLogger.php  EcomErroresLogger: handlers + escritura de logs  
│   └─ index.php  
├─ logs/  
│   ├─ .htaccess        Bloqueo de descarga  
│   └─ index.php  
├─ index.php  
├─ DOC_TECNICO.md  
├─ DOC_DESARROLLO.md  
└─ DOC_USUARIO.md
```

Flujo de arranque

1. PrestaShop instancia `EcomErrores` → el constructor llama `activarCaptura()`.
2. `activarCaptura()` lee la Configuration y llama `EcomErroresLogger::init($config)`.
3. `init()` registra los 3 handlers una sola vez (`self::$registered`).
4. Los hooks `actionDispatcherBefore` , `actionFrontControllerSetMedia` , `displayBackOfficeHeader` vuelven a llamar `activarCaptura()` como garantía (idempotente).

Puntos de extensión

- **Añadir campos al contexto:** ampliar `EcomErroresLogger::cartContext()` (siempre defensivo, con `is_object / is_callable` + try/catch).
- **Nuevos patrones de ignorado:** editables desde el back-office; internamente `ignorePatterns()` los parsea línea a línea.
- **Integrar avisos externos** (email/Slack): hacerlo dentro de `record()` **respetando la regla crítica de no envíos masivos** — aquí solo se dispara ante un error real y puntual del request en curso, nunca en bucle sobre histórico.

Uso manual desde otro código

```
require_once _PS_MODULE_DIR_ . 'ecomErrores/classes/ErrorLogger.php';  
EcomErroresLogger::writeManual('DEBUG', 'Mensaje que quiero volcar al log');
```


Decisiones de diseño

- **Encadenar y no reemplazar:** `set_exception_handler / set_error_handler` devuelven el handler previo (el de PrestaShop). Se guarda y se invoca tras loguear, para no romper la página 500 ni el modo dev de PS.
- **Excepciones antes que shutdown:** en PS 8/9 casi todos los "fatales" son `Throwable` con traza. El shutdown queda como red de seguridad para fatales sin traza.
- **Atribución por traza:** `pickOrigin()` + `detectModule()` saltan los frames de `var/cache` para señalar el módulo real, cumpliendo el requisito de ignorar tpl/cache/var sin perder la pista del culpable.
- **Fail-safe total:** cualquier fallo interno del logger se traga con try/catch doble (`Exception` y `Throwable`). El logger jamás debe agravar el problema que investiga.

Pruebas recomendadas

1. Activar el módulo, abrir la config y pulsar **Escribir prueba** → aparece línea en el log.
2. Reproducir el fallo con carrito → revisar `logs/error-YYYY-MM-DD.log`.
3. Verificar que un `.tpl`/cache no genera entradas (nivel `fatal_exceptions`).
4. Con nivel `all`, confirmar que aparecen warnings/notices (solo para depurar, luego bajar nivel).

Estándares aplicados

Cabecera AFL-3.0 en todos los ficheros, `if (!defined('_PS_VERSION_')) exit;`, `isUsingNewTranslationSystem()=true`, sin Composer, sin namespaces en el principal, rutas vía `_PS_MODULE_DIR_`, `HelperForm` con `fields_value` completo.